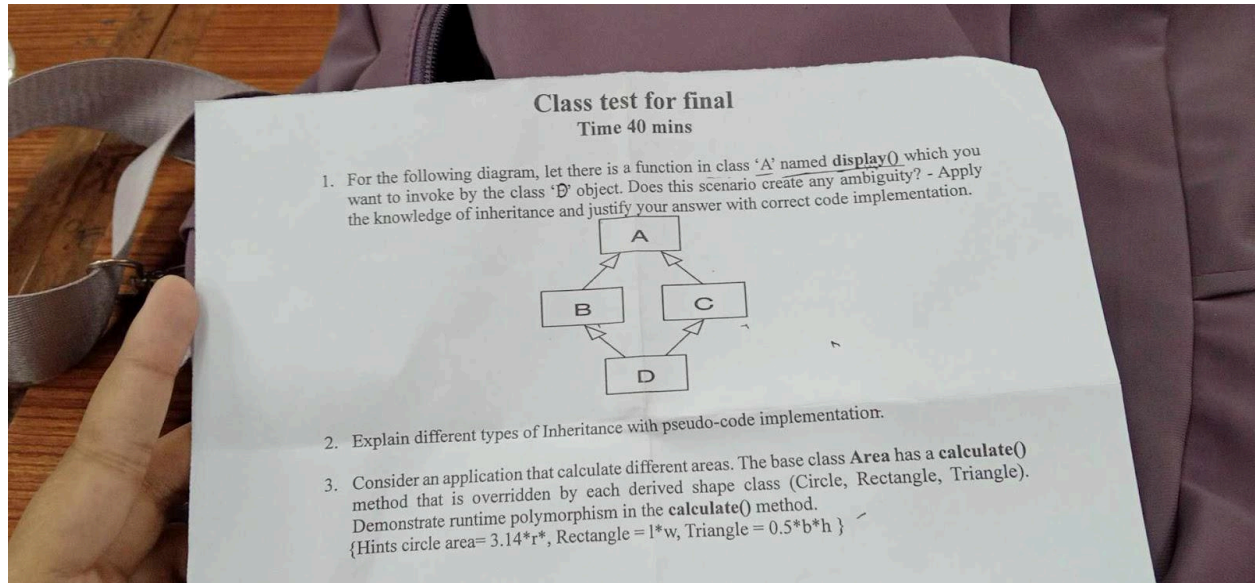


Class test for final 1



Question 1: Diamond Problem

The question asks whether there's any ambiguity when invoking the **display()** function from class A using an object of class D in the following inheritance hierarchy:



```
#include <iostream>
using namespace std;
class A{
public:
    virtual void Display(){
        cout << "Value of A" << endl;
    }
};
// Parent class 1 with virtual inheritance
class B: virtual public A{
```

```

};
// Parent class 2 with virtual inheritance
class C: virtual public A{

};
// Child class inheriting from both Parent1 and Parent2
class D: public B, public C{

};
int main() {
    D d;
    d.Display();

    return 0;
}

```

Question 2: Explain different types of Inheritance with pseudo- code implementation.

Type of inheritance

1: Single inheritance :a class is allowed to inherit from only one class.

Example:

```

Class A
|
Class B

```

```

class A
{
... ..
};
class B: public A
{
... ..
}

```

2. One child class inherits from multiple parent classes

Example:

Class A
|
Class B
|
Class C

```
class C
{
... ..
};
class B : public C
{
... ..
};
class A: public B
{
... ..
};
```

3. Multiple inheritance: one child class inheritance from multiple parent class;

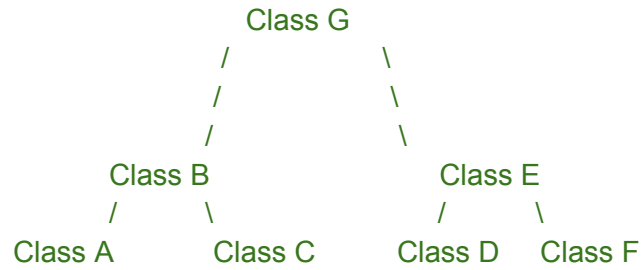
Example:

(base class 1) Class A Class B (base class 2)
 \
 / Class C (derived class)

```
class B
{
... ..
};
class C
{
... ..
};
class A: public B, public C
{
... ..
};
```

4. Hierarchical inheritance: Multiple child class inheritance from a parent class;

Example:

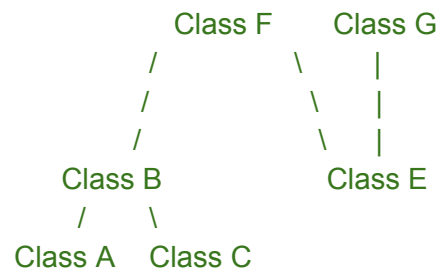


```

class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
  
```

5 Hybrid inheritance: Combination of two or more type of inheritance classes;

Example:



```

class F
{
    ....
}
  
```

```

class G
{
... ..
}
class B : public F
{
... ..
}
class E : public F, public G
{
... ..
}
class A : public B {
... ..
}
class C : public B {
... ..
}

```

Question 3:

3. Consider an application that calculate different areas. The base class **Area** has a **calculate()** method that is overridden by each derived shape class (Circle, Rectangle, Triangle). Demonstrate runtime polymorphism in the **calculate()** method.
{Hints circle area= $3.14 * r^2$, Rectangle = $l * w$, Triangle = $0.5 * b * h$ }

```

#include <iostream>
using namespace std;
class Area{
public:
virtual void Calculate(){
}
};
class Circle: public Area{
public:
double r;
Circle(double R): r(R){}
void Calculate(){
cout << 3.1416 * r << endl;
}
};

class Rectangle: public Area{

```

```

    public:
    double l , w;
    Rectangle( double L, double W): l(L), w(W) {}
    void Calculate(){
        cout << l * w << endl;
    }

};

class Triangle: public Area{
    public:
    double b , h;
    Triangle( double B, double H): b(B), h(H) {}
    void Calculate(){
        cout << 0.5 * b * h << endl;
    }
};

int main() {
    Circle C(5);
    C.Calculate();
    Rectangle R(5,6);
    R.Calculate();
    Triangle T(5, 6);
    T.Calculate();

    return 0;
}

```